

# Masterclass: Vistas, Estilos y Renderizado en React Native

Guía Exhaustiva y Estructurada para Exámenes

## Índice

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>I Elementos Básicos de Interfaz</b>                        | <b>3</b>  |
| 1. La Filosofía “Caja sobre Caja”: <View>                     | 3         |
| 2. Anatomía de los Textos y Variables                         | 3         |
| 2.1. Tipos de Texto . . . . .                                 | 3         |
| 2.2. Inyectando Datos de la Base de Datos . . . . .           | 3         |
| <b>II Posicionamiento y Layouts (Flexbox)</b>                 | <b>5</b>  |
| 3. La Gravedad por Defecto (Columna)                          | 5         |
| 4. Cambiando la Gravedad: flexDirection                       | 5         |
| 5. Alineación Fina: Justify y Align                           | 5         |
| 5.1. Eje Horizontal (justifyContent) . . . . .                | 5         |
| 5.2. Eje Vertical (alignItems) . . . . .                      | 5         |
| 6. Ejemplo Integrador: Tarjeta Completa                       | 6         |
| <b>III Interactividad y Listas Optimizadas</b>                | <b>7</b>  |
| 7. Botones Reactivos: <Pressable>                             | 7         |
| 8. El Motor Automático: <FlatList>                            | 7         |
| <b>IV Renderizado Inteligente (Lógica Dinámica)</b>           | <b>8</b>  |
| 9. Técnica 1: El Operador Ternario (Plan A o Plan B)          | 8         |
| 10. Técnica 2: El Operador AND (&&) (El Renderizado Fantasma) | 8         |
| 11. Técnica 3: Pantallas Vacías (Empty States)                | 8         |
| <b>V Entrenamiento para Exámenes</b>                          | <b>10</b> |
| 12. Pasos para Resolver Cualquier Pantalla                    | 10        |



# Parte I

## Elementos Básicos de Interfaz

*En esta sección aprenderemos los ladrillos fundamentales: las cajas y los textos que van dentro de ellas.*

### 1. La Filosofía “Caja sobre Caja”: <View>

Cuando diseñas en React Native, debes olvidar el diseño gráfico tradicional. Aquí no dibujamos elementos libres en un lienzo; aquí apilamos y organizamos **cajas**.

El <View> es el contenedor por excelencia. Por sí mismo es invisible, pero es vital porque funciona como:

- **Agrupador lógico:** Junta un título con su subtítulo para que no se separen.
- **Estructurador:** Sirve para crear filas y columnas.
- **Tarjeta visual:** Cuando le das estilos (bordes, sombras, color de fondo).

#### Regla de Oro del Examen

Nunca dejes un componente flotando solo. Si tienes varios textos juntos o un texto al lado de un botón, **siempre** envuélvelos en un <View> para poder controlarlos.

### 2. Anatomía de los Textos y Variables

En proyectos reales (como DeliverUS), no verás el componente genérico <Text>. Se usan envoltorios personalizados para unificar el diseño.

#### 2.1. Tipos de Texto

- **<TextRegular>:** Texto estándar. Usado para descripciones, calles, precios.
- **<TextSemiBold>:** Texto en negrita. Usado para Títulos, Nombres de entidades o Alias.
- **<TextError>:** Texto rojo. Usado estrictamente para mensajes de validación en formularios.

#### 2.2. Inyectando Datos de la Base de Datos

La verdadera potencia es mezclar diseño con datos usando las llaves { }.

```
1 // MAL: Los textos se pegaran, resultando en "AvenidaSevilla"  
2 <TextRegular>{item.street}{item.city}</TextRegular>  
3  
4 // BIEN: Intercalamos texto fijo (comas y espacios) fuera de las llaves  
5 <TextRegular>{item.street}, {item.city} ({item.postalCode})</TextRegular>
```

## Parte II

# Posicionamiento y Layouts (Flexbox)

*Ya tenemos cajas y textos. Ahora vamos a aprender a ordenarlos en la pantalla.*

### 3. La Gravedad por Defecto (Columna)

Si no aplicas ningún estilo, los elementos dentro de un `<View>` “caen” uno debajo del otro, de arriba a abajo.

```
1 <View>
2   <TextSemiBold>Mi Casa</TextSemiBold>
3   <TextRegular>Avenida Reina Mercedes, Sevilla</TextRegular>
4 </View>
```

*Visualización: El título "Mi Casa."parecerá justo encima de la calle.*

### 4. Cambiando la Gravedad: `flexDirection`

Para colocar cosas **una al lado de la otra** (como una calle a la izquierda y un botón a la derecha), necesitas usar el modificador `flexDirection: 'row'`.

```
1 <View style={{ flexDirection: 'row' }}>
2   <TextRegular>Izquierda</TextRegular>
3   <TextRegular>Derecha</TextRegular>
4 </View>
```

### 5. Alineación Fina: `Justify` y `Align`

Una vez declaras `flexDirection: 'row'`, usas estas dos reglas para controlar la posición exacta:

#### 5.1. Eje Horizontal (`justifyContent`)

- **center:** Lo agrupa todo en el centro de la pantalla.
- **space-between: Crucial.** Empuja el bloque izquierdo al borde izquierdo, y el bloque derecho al borde derecho. Ideal para separar el contenido de los botones de acción.

#### 5.2. Eje Vertical (`alignItems`)

- **center: Crucial.** Centra los elementos verticalmente para que un texto pequeñito y un icono grande queden perfectamente alineados en el medio.

## 6. Ejemplo Integrador: Tarjeta Completa

```
1  const renderUserCard = () => (  
2    // CAJA PRINCIPAL (La fila entera)  
3    <View style={{ flexDirection: 'row', justifyContent: 'space-between', alignItems:  
      'center' }}>  
4  
5      // CAJA IZQUIERDA (Agrupa textos, caen en columna por defecto)  
6      <View>  
7        <TextSemiBold>Julio Rodriguez</TextSemiBold>  
8        <TextRegular>julio@ejemplo.com</TextRegular>  
9      </View>  
10  
11     // CAJA DERECHA (Boton)  
12     <View>  
13       <Pressable onPress={borrarUsuario}>  
14         <Ionicons name="trash" size={24} color="red" />  
15       </Pressable>  
16     </View>  
17  
18   </View>  
19 );
```

## Parte III

# Interactividad y Listas Optimizadas

*Vamos a dar vida a nuestros elementos y a generar listas de forma eficiente.*

## 7. Botones Reactivos: <Pressable>

El <Pressable> envuelve cualquier cosa que queramos que el usuario pueda tocar. Su gran ventaja es que nos permite cambiar su estilo *justo en el milisegundo* en que el usuario tiene el dedo pegado a la pantalla.

En lugar de pasarle un estilo normal, le pasamos una función que lee la variable **pressed**:

```
1 <Pressable
2   onPress={() => eliminarObjeto(item.id)}
3   style={({ pressed }) => [
4     styles.botonGenerico, // Las reglas fijas (bordes, márgenes)
5     {
6       // Dinámico: Si estoy tocando (pressed) me pongo gris, si no, transparente
7       backgroundColor: pressed ? '#ddddd' : 'transparent'
8     }
9   ]}
10 >
11   <Icons name="trash" size={24} color="red" />
12 </Pressable>
```

## 8. El Motor Automático: <FlatList>

Nunca renderizamos arrays enteros a mano (usando `.map()`). Usamos <FlatList> porque solo dibuja lo que cabe en la pantalla, ahorrando memoria. Exige 3 propiedades clave:

1. **data**: La variable de tu `useState` que contiene el array.
2. **keyExtractor**: El identificador único de cada fila (`item => item.id.toString()`).
3. **renderItem**: La función "molde" que diseñamos arriba.

```
1 // Dentro del return principal de tu componente:
2 return (
3   <View style={styles.pantalla}>
4     <FlatList
5       data={misDirecciones}
6       renderItem={renderDireccionMolde} // Llama a la función que tiene las cajas
7       keyExtractor={(item) => item.id.toString()}
8     />
9   </View>
10 );
```

# Parte IV

## Renderizado Inteligente (Lógica Dinámica)

*Cómo hacer que la interfaz cambie de forma automática según la información que llegue de la base de datos.*

### 9. Técnica 1: El Operador Ternario (Plan A o Plan B)

Se usa cuando necesitas mostrar un diseño u otro obligatoriamente, dependiendo de una condición.

**Ejemplo: Estrella favorita vs Estrella normal**

```
1 <Pressable onPress={() => marcarFavorito(item)}>
2   <Icons
3     // Si isDefault es true, pinta 'star'. Si es false, pinta 'star-outline'.
4     name={item.isDefault ? 'star' : 'star-outline'}
5     size={24}
6     color="gold"
7   />
8 </Pressable>
```

### 10. Técnica 2: El Operador AND (&&) (El Renderizado Fantasma)

Se usa cuando quieres añadir un elemento extra **solamente** si se cumple una condición. Si no se cumple, no se dibuja nada.

**Ejemplo: Cartel condicional**

```
1 <View>
2   <TextSemiBold>{item.name}</TextSemiBold>
3
4   {/* El cartel de "Agotado" solo existe en el movil si el stock es 0 */}
5   {item.stock === 0 && (
6     <TextError>Producto Agotado temporalmente</TextError>
7   )}
8 </View>
```

### 11. Técnica 3: Pantallas Vacías (Empty States)

Si la base de datos nos devuelve una lista vacía, es muy feo mostrar una pantalla en blanco. Debemos interceptar el `<FlatList>` usando un ternario a nivel gigante.



```

1 return (
2   <View style={styles.container}>
3     {pedidos.length === 0 ? (
4       // PLAN A: Array vacio -> Mostramos mensaje de aviso
5       <View style={styles.emptyContainer}>
6         <TextSemiBold>Aun no tienes pedidos guardados.</TextSemiBold>
7       </View>
8     ) : (
9       // PLAN B: Array con datos -> Mostramos la lista
10      <FlatList
11        data={pedidos}
12        renderItem={renderPedido}
13        keyExtractor={item => item.id.toString()}
14      </>
15    )}
16  </View>
17 );

```

## Parte V

# Entrenamiento para Exámenes

### 12. Pasos para Resolver Cualquier Pantalla

Sigue este guion de 5 pasos en el examen para no bloquearte nunca:

1. **Analiza los datos:** Mira tu `useState` o el JSON del backend. ¿Qué variables trae el `item`? (`name`, `price`, `address`).
2. **Diseña en papel:** Dibuja la fila. Rodea en un rectángulo gigante la fila entera. Luego, haz un rectángulo a la izquierda (textos) y otro a la derecha (botones).
3. **Construye el Molde (`renderItem`):** Traduce tus rectángulos de papel a código usando `<View>`. Ponles `flexDirection: 'row'` donde sea necesario.
4. **Injecta variables:** Rellena tus componentes `<Text...>` con `{item.loQueSea}`. Añade ternarios si te piden que algo cambie de color.
5. **Conecta el `FlatList`:** Ve a tu `return` principal, monta el `<FlatList>` y pásale la función molde que acabas de crear.

### 13. Ejercicio de Traducción Mental

**Enunciado del examen:** *Cree una lista de restaurantes. Cada restaurante debe mostrar su logo a la izquierda. A su derecha, el nombre del restaurante y la categoría. Al final de la fila, el tiempo medio de envío.*

**Estructura mental a generar:**

```
1  const renderRestaurante = ({ item }) => (  
2    // 1. Rectangulo Principal (row, space-between, align center)  
3    <View style={styles.filaRestaurante}>  
4  
5      // 2. Rectangulo Izquierdo (row, align center)  
6      <View style={styles.bloqueIzquierdo}>  
7        <Image source={{uri: item.logo}} style={styles.imagen} />  
8  
9        // 3. Rectangulo de Textos (column por defecto, sin estilos extra)  
10       <View style={{ marginLeft: 10 }}>  
11         <TextSemiBold>{item.name}</TextSemiBold>  
12         <TextRegular>{item.categoryName}</TextRegular>  
13       </View>  
14     </View>  
15  
16     // 4. Rectangulo Derecho (Tiempo de envio)  
17     <View>  
18       <TextRegular>{item.shippingTime} min</TextRegular>  
19     </View>  
20  )
```

```
21   </View>
22 );
```